



# Automatic Prompt Engineering

## Automatic Prompt Engineering (APE)

### 1) What it is

**Automatic Prompt Engineering (APE)** is a workflow where an LLM (or small ensemble) **generates, evaluates, and iteratively improves prompts** with minimal human effort. Instead of hand-crafting a single prompt, you:

1. auto-generate multiple prompt variants,
2. score them against a **dev set** with objective metrics,
3. select (and optionally mutate) the best, and
4. repeat until performance stabilizes.

#### Why it matters

- **Scale:** Explore far more variants than manual trial-and-error.
  - **Quality:** Finds phrasing humans might not consider.
  - **Speed/Cost control:** Automated sweeps reveal “good enough” prompts for given budgets.
  - **Transferability:** Discovered patterns can be templated across tasks.
- 

### 2) Core APE loop (overview)

#### 1. Seed

- Define *task spec* (inputs, desired outputs, constraints, success metric).
- Provide *few strong exemplars* or a rubric.

#### 2. Generate

- Use an LLM to produce  $N$  prompt candidates (e.g., 10–100).

- Optionally vary decoding params (temperature, top-p) and include *mutations* (role changes, ordering, framing).

### 3. Evaluate

- Run each candidate on a **development set**.
- Compute metrics (task-specific).

### 4. Select & Mutate

- Keep top-k prompts.
- Mutate: rephrase instructions, swap examples, adjust constraints, or insert tool/format hints.

### 5. Stop or Iterate

- Stop when metric plateaus or budget exhausted; otherwise loop back to **Generate**.

---

## 3) Metrics (choose task-appropriate)

### Exact/structured tasks

- **Accuracy / F1 / Exact Match** — classification, slot filling, extraction.
- **Schema validity rate** — % valid JSON/XML (for structured outputs).
- **Constraint adherence** — length, sections present, key fields.

### Generative tasks

- **ROUGE / BLEU** — n-gram overlap (useful but imperfect).
- **BERTScore / BLEURT** — semantic similarity.
- **Edit distance** — compression/summarization tightness.
- **LLM-as-a-Judge** — rubric-based scoring (pairwise or scalar). *Mitigate bias with multiple judges + majority vote.*

### Operational

- **Latency & cost per sample, token usage, error rate, safety flags** (toxicity, PII).

Tip: Combine automatic metrics + small human audit for fidelity. For pairwise comparisons, track Elo/TrueSkill to rank prompts robustly.

## 4) Example (classification) — “customer order intent”

**Task:** Given a user message, label `{PlaceOrder|ModifyOrder|CancelOrder|Status|Other}`.

**Dev set:** 300 labeled messages (balanced).

**APE plan:**

1. **Generate** 25 prompt variants with different role frames (e.g., “retail CSR”), constraint language, and ordering of labels.
2. **Evaluate** on the dev set: **macro F1**, **exact match**, and **confusion matrix**.
3. **Select** top 5; **mutate** the best 2 with added negative examples (“don’t infer product if unspecified”).
4. **Re-test**; pick champion; **hold-out** test on a separate 200-sample set.
5. **Shadow deploy** to 10% of traffic; monitor live KPIs (handoffs to human, CSAT, recontact rate).

**Sample prompt candidate (structured)**

System: You are a retail customer-support classifier. Return ONLY valid JSON.

User: Given a customer message, classify into one of:

- PlaceOrder
- ModifyOrder
- CancelOrder
- Status
- Other

Rules:

- Use only the message content.
- If you cannot infer, return "Other".
- Provide one label and brief rationale.

Output JSON schema:

```
{ "label": "PlaceOrder|ModifyOrder|CancelOrder|Status|Other", "rationale": string }
```

Message: "<<MSG>>"

---

## 5) Controller pseudo-code (offline sweep)

```
# Pseudocode (controller)
prompts = llm_generate_variants(seed_instructions, n=50)
# Generation step
results = []

for p in prompts:
    preds = run_prompt_on_dataset(p, dev_examples)
# Batch inference
    metrics = evaluate(preds, dev_labels, task="classification",
                       extra={"schema_validity": check_json(
                           preds)})
    results.append({"prompt": p, "metrics": metrics})

top = select_top_prompts(results, by="macro_f1", k=5)

mutations = mutate_prompts([t["prompt"] for t in top],
                            ops=["reorder_labels", "tighten_rules", "add_neg_examples"])
second_round = prompts_from(top) + mutations

# Iterate or finalize; persist artifacts + seeds for reproducibility
save_experiment(prompts, results, metadata={"model": "gpt-4o", "temp": 0.2})
```

---

## 6) Prompt-variant generation patterns

- **Role swaps:** "You are a Tier-2 CSR" vs "You are a compliance auditor."
- **Instruction style:** bullet rules vs numbered steps.
- **Ordering:** constraints first, then examples (often improves adherence).
- **Explicit failure policy:** "If uncertain, output Other/null."
- **Format enforcement:** JSON/grammar mode; add `stop` sentinels.

- **Few-shot toggles:**  $0 \rightarrow 1 \rightarrow 3$  examples; vary *counter-examples* (negative demonstrations).
  - **Decoding knobs:** low temperature for determinism in structured tasks; moderate for creative tasks.
- 

## 7) Beyond BLEU/ROUGE (limitations & fixes)

- N-gram metrics ignore *faithfulness* and *reasoning validity*.
  - For summaries/explanations, add **factuality checks** (entity overlap, citation presence) or **reference-free** metrics (QAFactEval-like, or LLM-judge with a sourced rubric).
  - Use **pairwise preference** judgments with strict rubrics ("Which summary is more factual and complete? Cite errors."); average across **multiple judges/seeds**.
- 

## 8) Guardrails & reliability

- **Schema enforcement:** JSON/grammar modes; validate with JSON Schema/Zod.
  - **Stop sequences:** crisp boundaries for multi-part outputs.
  - **Safety checks:** toxicity/PII detectors; blocklists/allowlists.
  - **Determinism:** low temperature + fixed **seed** (if provider supports); stable model version pinning.
  - **Data hygiene:** prevent **train-test leakage**; keep hold-out & temporal splits.
  - **Observability:** log prompt version, params, model, latency, tokens, stop reason.
- 

## 9) Search strategies (optimize faster)

- **Grid/Random search** over small spaces (cheap, baseline).
- **Bandits** (epsilon-greedy, UCB) for online selection with live traffic.
- **Bayesian optimization** (e.g., Gaussian Processes) on continuous params (temp, top-p) and discrete design choices (sections on/off).

- **Evolutionary algorithms** for prompt text: crossover of top performers; mutation operators that rephrase constraints.
- 

## 10) Human-in-the-loop (HITL)

- **Rubric drafting:** humans define success criteria & counter-examples.
  - **Audit small samples:** verify factuality, tone, harm avoidance.
  - **Active error collection:** add worst failures back into the dev set.
  - **Red teaming:** adversarial inputs (jailbreak attempts, edge cases).
- 

## 11) Deployment patterns

- **Champion/Challenger:** production uses Champion; evaluate 1–2 Challengers on a traffic slice.
  - **Canary & rollback:** automatic rollback if adherence/safety dips.
  - **Drift monitoring:** watch input distribution; re-run APE periodically.
- 

## 12) Cost & budget tips

- Start with **small dev sets** (100–300) to prune bad prompts.
  - Use **cheaper models** for coarse filtering; re-score finalists on your top model.
  - Cache model calls; de-duplicate identical inputs.
  - Cap **max\_tokens**; prefer structured outputs to minimize verbosity.
- 

## 13) Reusable artifacts (copy-paste)

### Evaluation rubric (LLM-judge)

Score 1–5 on:

- 1) Task fidelity (followed instructions exactly?)
- 2) Output format correctness (valid JSON? required keys?)
- 3) Factuality/grounding (no invented details)
- 4) Helpfulness/coverage (addresses all parts)

5) Style constraints (tone, length)

Return JSON: {"score": number, "notes": string}

### Prompt record schema

```
{
  "id": "ape-2025-08-19-001",
  "model": "gpt-4o",
  "temperature": 0.2,
  "prompt_text": "...",
  "examples": [...],
  "metrics": {"macro_f1": 0.87, "schema_validity": 0.99},
  "cost_tokens": {"input": 123456, "output": 98765},
  "createdAt": "2025-08-19T00:00:00Z",
  "version": "1.3.0"
}
```

### Mutation operator templates

- Tighten: “If uncertain, return 'Other'; do NOT infer missing fields.”
- Structure: “Return ONLY valid JSON. Keys: {...}. No prose.”
- Ordering: Put constraints first, then examples, then the new input.
- Negative exemplars: Include 1–2 tricky cases showing what NOT to do.

## 14) Worked micro-example (10 variants → top pick)

- **Generate** 10 variants that differ in role, constraint phrasing, and example ordering.
- **Evaluate** on 120 labeled messages.
- **Result:** top variant improves macro-F1 from **0.82** → **0.88**, schema validity **96%** → **100%**, cost +7%.
- **Decision:** ship as **Champion**; keep previous best as **Challenger**.

## 15) Key takeaways

- APE is **search + measurement + iteration**; treat prompts like code artifacts.
- Use **task-appropriate metrics**; ROUGE/BLEU are not enough for many tasks.
- Enforce **structure** (JSON/grammar, stop sequences) and **validate** outputs.
- Combine automated scoring with **HITL** for fidelity and safety.
- Instrument everything: **version, log, test, and monitor** to keep improvements real and durable.